
Ejemplo del Perceptrón

Introducción

El perceptrón es un algoritmo de aprendizaje simple diseñado por Frank Rosenblatt. Entraremos en el formalismo en un momento. Primero considera un ejemplo extremo: un conjunto de datos que contiene una lista de pesos y las etiquetas binarias correspondientes que dicen elefante o ratón dependiendo del peso.

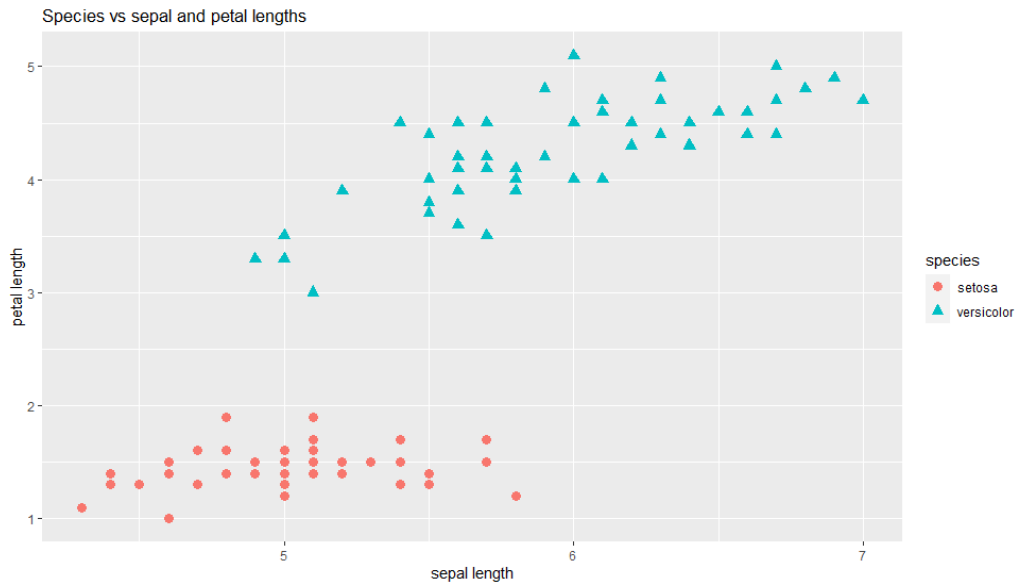
El perceptrón tomaría un conjunto de datos de entrenamiento y eventualmente aprendería a identificar las clases correctas en función del peso del objeto. Por supuesto, ciertas condiciones deben cumplirse para las propiedades de convergencia del modelo.

Perceptrón - Algoritmo de clasificación binaria

Aquí, usamos el conjunto de datos de Iris.

```
> # cargar conjunto de datos de iris
> data(iris)
> # subconjunto del marco de datos del iris: extraer solo especies versicolor y setosa
> # solo nos centraremos en las longitudes de sépalos y pétalos del conjunto de datos
> irissubdf <- iris[1:100, c(1, 3, 5)]
> names(irissubdf) <- c("sepal", "petal", "species")
> head(irissubdf)
  sepal petal species
1  5.1   1.4  setosa
2  4.9   1.4  setosa
3  4.7   1.3  setosa
4  4.6   1.5  setosa
5  5.0   1.4  setosa
6  5.4   1.7  setosa

> # trazar datos: una imagen vale más que 1000 palabras. melt data => luego ggplot
> library(ggplot2) # no olvides instalar primero este paquete install.packages("ggplot2")
> ggplot(irissubdf, aes(x = sepal, y = petal)) +
+   geom_point(aes(colour=species, shape=species), size = 3) +
+   xlab("sepal length") +
+   ylab("petal length") +
+   ggtitle("Species vs sepal and petal lengths")
```



Agregar etiquetas binarias correspondientes a las especies - Inicializar todos los valores a 1
 Agrega la etiqueta setosa de -1. Las etiquetas binarias +1, -1 están en la cuarta columna. Es mejor crear dos marcos de datos separados: uno que contenga los atributos mientras que el otro contiene los valores de clase.

```
> irissubdf[, 4] <- 1
> irissubdf[irissubdf[, 3] == "setosa", 4] <- -1
> x <- irissubdf[, c(1, 2)]
> y <- irissubdf[, 4]
> # cabeza y cola de datos
> head(x)
  sepal petal
1  5.1  1.4
2  4.9  1.4
3  4.7  1.3
4  4.6  1.5
5  5.0  1.4
6  5.4  1.7
> head(y)
[1] -1 -1 -1 -1 -1 -1
```

En esta sección, simplemente implementaremos el algoritmo de aprendizaje del perceptrón para estos datos. En la siguiente sección, trabajaremos en la convergencia de los factores de peso y jugaremos con la tasa de aprendizaje.

Escribimos una función que toma el marco de datos, la tasa de aprendizaje - eta y el número de épocas - n.iter y actualiza el factor de peso. En esta etapa, solo nos preocupa el peso final y el número de épocas requeridas para que el peso converja.

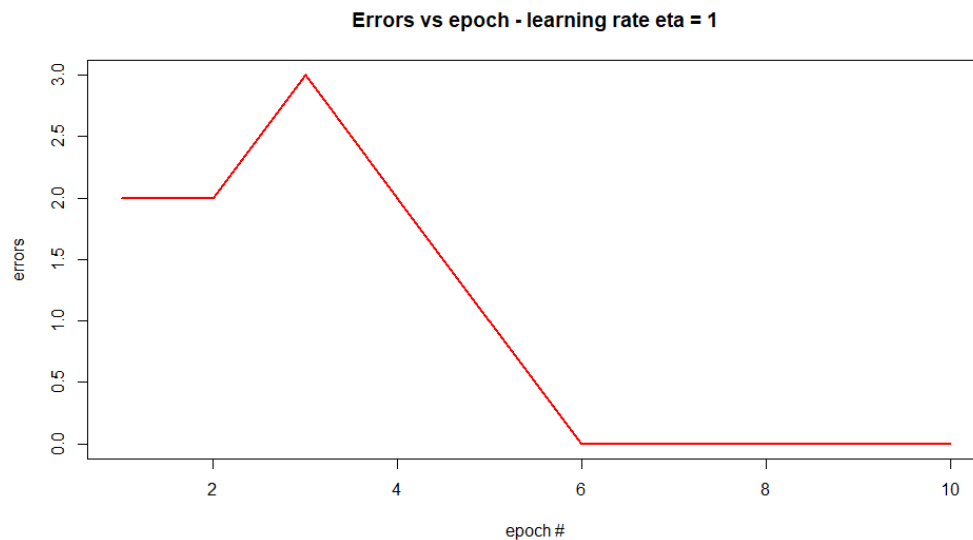
Es más interesante cargarlo desde un archivo (no olvides que por simplicidad este archivo se encuentra en la carpeta de documentos de windows).

```
> source('perceptron.R')
```

```
[1] -4.0 -6.8 18.2
```

```
> plot(1:10, err, type="l", lwd=2, col="red", xlab="epoch #", ylab="errors")
```

```
> title("Errors vs epoch - learning rate eta = 1")
```



Ten en cuenta que **aumentar la tasa de aprendizaje hace exactamente lo que sugiere su nombre**, el único problema es que **la tasa puede no ser lo suficientemente sensible para garantizar la convergencia**. Si bien la convergencia está asegurada, **una mayor tasa de aprendizaje (entre 0 y 1) conduce a una convergencia más rápida**.

¿Cómo implementar una clasificación multiclase en el perceptrón?

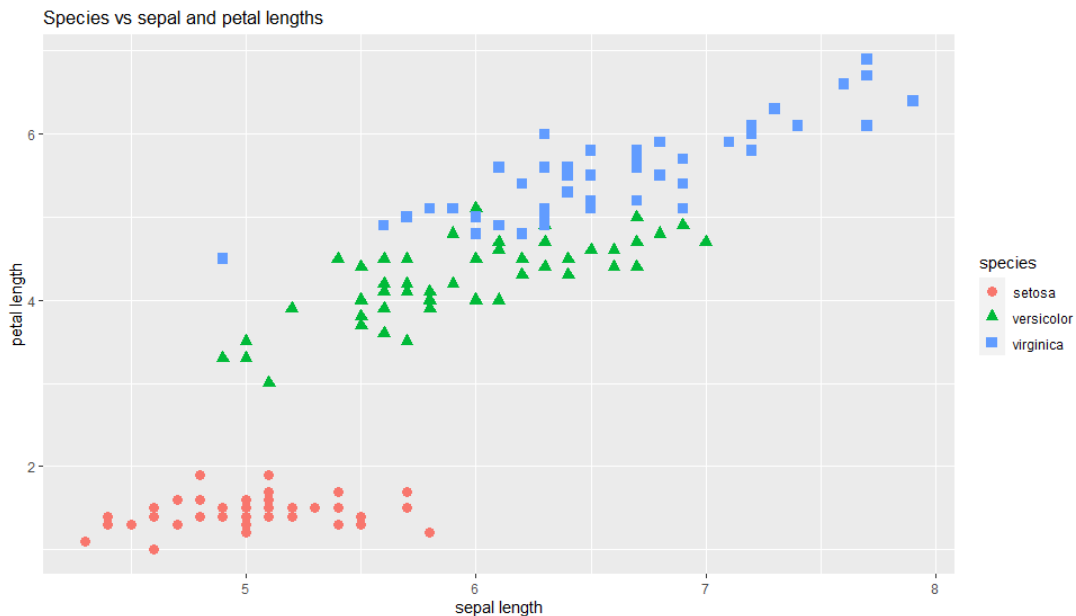
El conjunto de datos de iris con el que comenzamos tenía tres especies diferentes. Eliminamos Virginica por el bien de la clasificación binaria. Ahora, usando lógica simple, extenderemos el algoritmo del perceptrón para tratar con tres especies.

El plan de juego es el siguiente:

- Primero crea un marco de datos con longitudes de sépalos y pétalos.

- Etiqueta los datos de especies correspondientes como 1 para virginica y -1 para setosa O versicolor.
- El peso obtenido tras la convergencia decidirá entre virginica y no virginica.
- Luego, el peso de la sección anterior se puede usar para determinar entre setosa y versicolor.
- El posible problema que podría surgir es que la longitud de los sépalos y los pétalos podría no ser suficiente para clasificar la especie. Entonces, comienza por trazarlo.

```
> irisdata <- iris[, c(1, 3, 5)]
> names(irisdata) <- c("sepal", "petal", "species")
>
> ggplot(irisdata, aes(x = sepal, y = petal)) +
+   geom_point(aes(colour=species, shape=species), size = 3) +
+   xlab("sepal length") +
+   ylab("petal length") +
+   ggtitle("Species vs sepal and petal lengths")
```



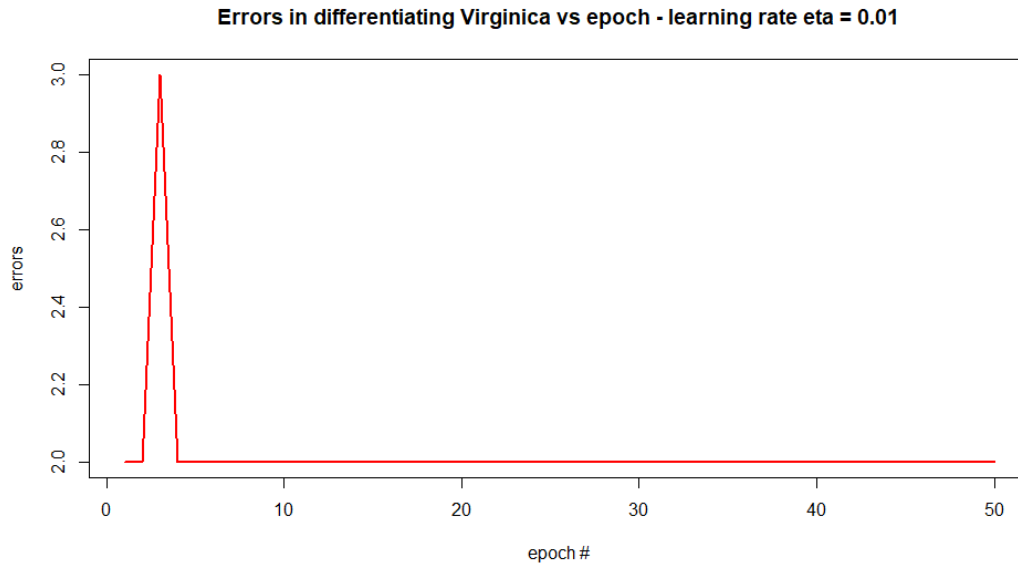
A partir de la gráfica, podemos ver que los datos no son linealmente separables en función de estos atributos. Se necesitan más atributos. Por lo tanto, mantendremos todos los atributos del conjunto de datos del iris (largos y anchos de sépalos y pétalos) para ver si el peso puede converger o no.

```
> # subconjunto de propiedades de las flores del conjunto de datos de iris
> x <- iris[, 1:4]
> names(x) <- tolower(names(x))
> # crear etiquetas de especies
```

```

> y <- rep(-1, dim(x)[1])
> y[iris[, 5] == "virginica"] <- 1
> # calcular y trazar el error
> err <- perceptron(x, y, 0.01, 50) # seria interesante cambiar los parametros y graficar el error
[1] -0.020 -0.732 -0.254 0.944 0.748
> # si cambias los parametros, por ejemplo mas epocas, no olvides cambiar el rango del plot
> plot(1:50, err, type="l", lwd=2, col="red", xlab="epoch #", ylab="errors")
> title("Errors in differentiating Virginica vs epoch - learning rate eta = 0.01")

```



Esta grafica es aún más informativa que la diferenciación entre setosa y versicolor. En primer lugar, ten en cuenta que aunque obtenemos una convergencia del peso, no converge a 0. El mínimo de errores que podemos obtener es 2. **Corrección: El número de errores es temporalmente constante. Esto no implica que los pesos sean constantes. De hecho, ellos no lo son.** Dejamos intacta la conclusión inicial errónea como recordatorio posterior de un tipo de pensamiento falaz. Curiosamente, ahora podemos asignar un nivel de confianza semi significativo a nuestras predicciones, ya que nos equivocamos en 0.0133333 de las predicciones sin ponderaciones.

Otro pensamiento que surge es que primero deberías separar la setosa de la no setosa, ya que es linealmente separable. Cualquier incertidumbre que exista está en el siguiente paso, donde tratamos de distinguir entre conjuntos de datos no separables linealmente. Si lo hacemos en orden inverso, entonces la incertidumbre se propagará y se hará mayor.